Working with Time Series Data

Zachary Tipton

2026-04-14

In preparation for the remainder of the course, we will be thinking about working with data that is arranged in time. To do so, we are going to practice working with dates in R.

```
library(ggplot2) # install.packages("ggplot2")
library(lubridate) # install.packages("lubridate")
```

```
Attaching package: 'lubridate'

The following objects are masked from 'package:base':

    date, intersect, setdiff, union
```

```
library(slider) # install.packages("slider")
```

```
Warning: package 'slider' was built under R version 4.5.2
```

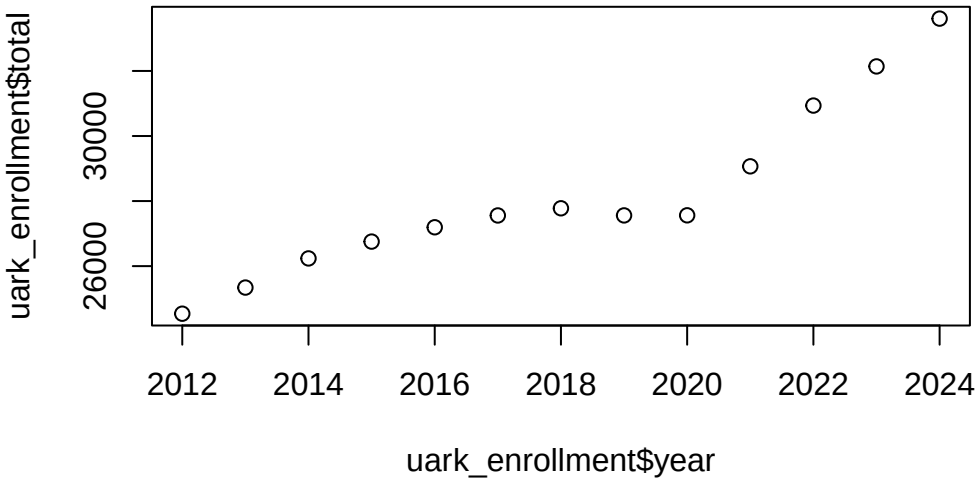
Years

The simplest time-series data to deal with is annual data. For example, take `uark_enrollment` below.

```
uark_enrollment <- data.frame(
  year = c(2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024),
  full_time = c(19508, 20379, 21047, 21415, 21668, 22144, 22602, 22193, 22070, 23282, 25214,
  part_time = c(5029, 4962, 5190, 5339, 5526, 5414, 5176, 5366, 5492, 5786, 5722, 3714, 3724)
)
uark_enrollment$total <-
  uark_enrollment$full_time + uark_enrollment$part_time
```

In this setting, year is just another regular *numeric* variable. Let's create a plot of enrollment over time. To do so, plot `year` on the x-axis and `total` on the y-axis.

```
plot(uark_enrollment$year, uark_enrollment$total)
```



What do you see?

Answer:

From 2012 to 2020 we're seeing a concave trend where enrollment marginally increases from 2012 to 2018 and then starts to decrease in 2019 and hold steady for 2020. We then see a sharp increase in enrollment each year from 2021 to 2024. Eyeballing it-the 2024 enrollment figure seems about 10,000 more than 2014.

Working with dates

However, when we get to dates (day month year), this gets more difficult. Here we have box scores from Arkansas football's 2023 season, but note the days are strings and not numbers. This makes things like plotting a little more difficult.

```
# Arkansas' 2023 football games
football <- data.frame(
  date = c(
    "11-11-2023", "11-04-2023", "09-23-2023", "09-02-2023", "10-07-2023",
    "09-16-2023", "09-09-2023", "10-21-2023", "11-24-2023", "10-14-2023",
    "11-18-2023", "09-30-2023"
  ),
  month = c(11, 11, 9, 9, 10, 9, 9, 10, 11, 10, 11, 9),
  day = c(11, 4, 23, 2, 7, 16, 9, 21, 24, 14, 18, 30),
```

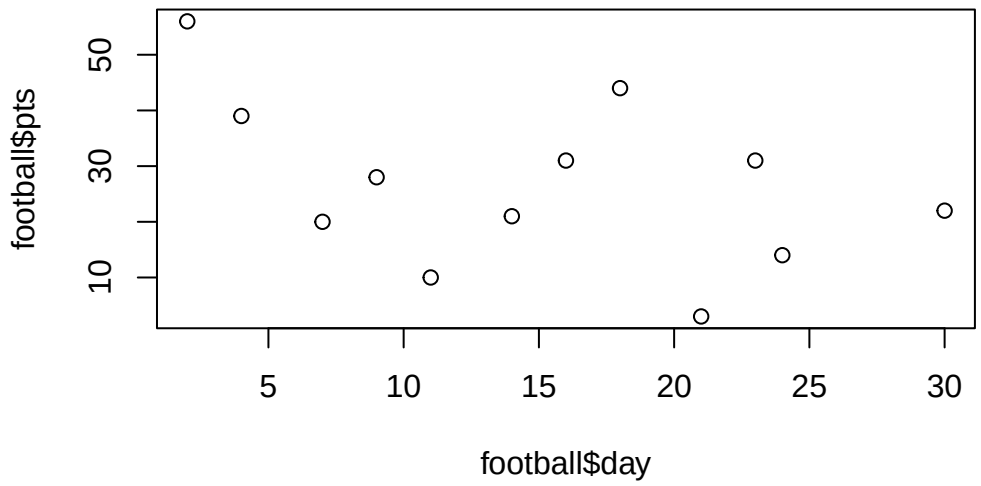
```
  year = rep(2023, 12L),
  school = rep("Arkansas", 12L),
  opponent = c(
    "Auburn", "Florida", "(12) LSU", "Western Carolina", "(16) Ole Miss", "BYU",
    "Kent State", "Mississippi State", "(10) Missouri", "(11) Alabama",
    "Florida International", "Texas A&M"
  ),
  result = c("L", "W", "L", "W", "L", "L", "W", "L", "L", "L", "W", "L"),
  pts = c(10, 39, 31, 56, 20, 31, 28, 3, 14, 21, 44, 22),
  pts_opponent = c(48, 36, 34, 13, 27, 38, 6, 7, 48, 24, 20, 34)
)
```

For example, say I wanted to plot the points scored by Arkansas over the season. Try that below, maybe with `date` or with `month`/`day`?

```
#plot(football$date, football$pts)
```

… It's not so easy to do this. For example, if I use `day` on the x-axis, these are not in the correct order.

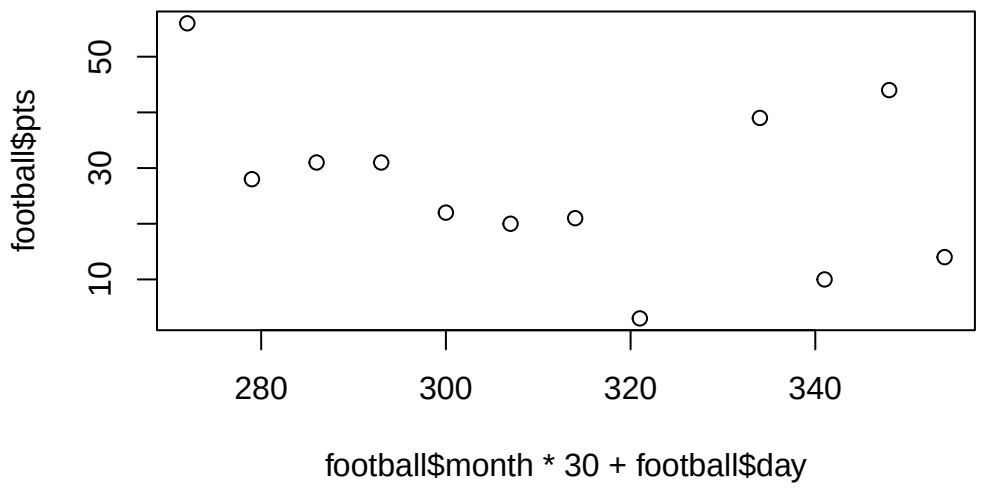
```
plot(football$day, football$pts)
```



If I try `date`, I just get an error since `football$date` is a string.

The best I could think to do is to kind of fake it by doing

```r
# approximately converts to days since January 1st
plot(football$month * 30 + football$day, football$pts)
```



Dates in R

R is a good language, so it turns out R has a bunch of functionality to work with dates. I think the easiest way to work with dates is to use the `lubridate` package which we loaded above

`lubridate` has a bunch of functions to help work with dates. First, we have `date()` which creates a Date object in R

```r
today <- Sys.Date()
print(today)
```

```
[1] "2026-04-14"
```

```r
class(today)
```

```
[1] "Date"
```

You can add and subtract days from `Date` objects. `1` is a single *day*.

```r
tomorrow <- today + 1
next_class <- today + 2
```

R knows leap years too!

```
date("2023-02-27") + 2
```

```
[1] "2023-03-01"
```

```
date("2024-02-27") + 2
```

```
[1] "2024-02-29"
```

`date` accepts a vector too:

```
last_4_classes <- date(c("2025-03-20", "2025-03-18", "2025-03-13", "2025-03-11"))
```

Aside: How dates are represented in R

Dates are internally represented as an integer in R. Try `as.numeric()` on the date and you'll get a weird number

```
today
```

```
[1] "2026-04-14"
```

```
as.numeric(today)
```

```
[1] 20557
```

It turns out, that the integer is the number of days since 1970-01-01 (an arbitrary reference point)

```
today - as.numeric(today)
```

```
[1] "1970-01-01"
```

```
today - date("1970-01-01")
```

```
Time difference of 20557 days
```

```
as.numeric(date("1970-01-01"))
```

```
[1] 0
```

```
as.numeric(date("1970-01-03"))
```

```
[1] 2
```

Back to football dataset

So returning to our previous problem, we can convert our string of dates to actual dates. But, if we try with `date`, we will get an error:

```
#date(football$date)
```

This is because the date is in an ambiguous format. It does not know if "11-04-2023" is November 4th or April 11th. (aside, year-month-day is the best format for this and other reasons !)

Instead, lurbidate has a set of functions like `mdy`, `dmy`, `ymd`, etc. that will work.

```
football$date <- mdy(football$date)
```

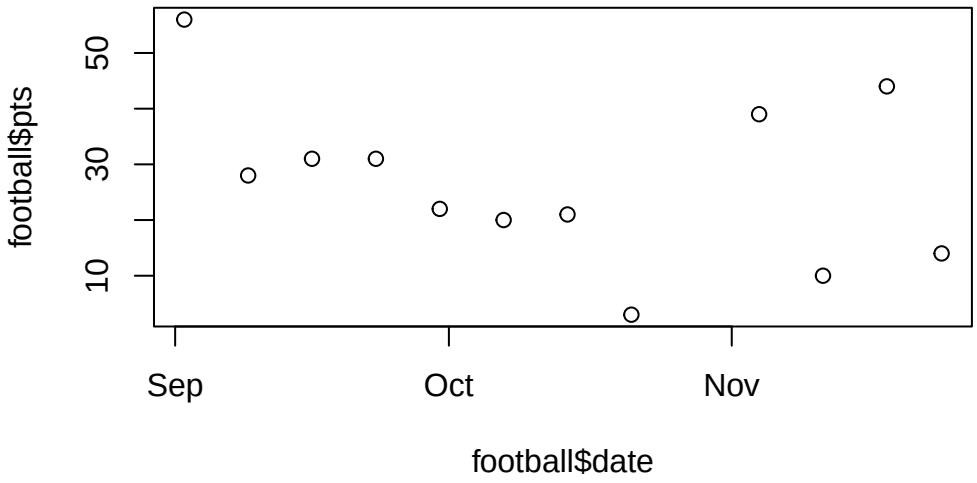
The function is good at recognizing various date formats, e.g.:

```
mdy("November 11th, 2023")
```

```
[1] "2023-11-11"
```

Now we can plot our scores over time. and look, R will print out pretty labels!!

```
plot(football$date, football$pts)
```



More `lubridate` functions

Okay, say we have a vector of `Date`s. I can use `lubridate`'s `year()`/`month()`/`day()` functions to extract the components.

Try the month function out on `football$date`. What happens if you add the argument `label = TRUE` option to `month`?

```
month(football$date)
```

```
 [1] 11 11  9  9 10  9  9 10 11 10 11  9
```

```
month(football$date, label = TRUE)
```

```
 [1] Nov Nov Sep Sep Oct Sep Sep Oct Nov Oct Nov Sep
12 Levels: Jan < Feb < Mar < Apr < May < Jun < Jul < Aug < Sep < ... < Dec
```

Another useful function is `wday` which gives you the day of week. See the help for more details. For an example, say we want to generate all the class days. We could create a sequence of days from the first to the last and then subset the vector based on the day of the week.

```
days <- seq(date("2025-01-14"), date("2025-05-01"), by = 1)
classes <- days[wday(days, label = TRUE) %in% c("Tue", "Thu")]
classes
```

```
 [1] "2025-01-14" "2025-01-16" "2025-01-21" "2025-01-23" "2025-01-28"
 [6] "2025-01-30" "2025-02-04" "2025-02-06" "2025-02-11" "2025-02-13"
[11] "2025-02-18" "2025-02-20" "2025-02-25" "2025-02-27" "2025-03-04"
[16] "2025-03-06" "2025-03-11" "2025-03-13" "2025-03-18" "2025-03-20"
[21] "2025-03-25" "2025-03-27" "2025-04-01" "2025-04-03" "2025-04-08"
[26] "2025-04-10" "2025-04-15" "2025-04-17" "2025-04-22" "2025-04-24"
[31] "2025-04-29" "2025-05-01"
```

Practice question

What is the most common month in the football dataset? Hint: use the `month` and `table` function for this.

```
table(month(football$date, label = TRUE))
```

```
Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
  0   0   0   0   0   0   0   0   5   3   4   0
```

September is the most common month.

Sorting by date

In this class, we will often need data to be sorted in order of time. The easiest way to do this is by using the `sort_by` function. The first argument to this function is the data.frame you want to sort and the second the vector you want to sort based on.

```
football <- sort_by(football, football$date)
# equivalent:
# football <- dplyr::arrange(football, date)
# equivalent:
# football <- football[order(football$date), ]
```

Unemployment data

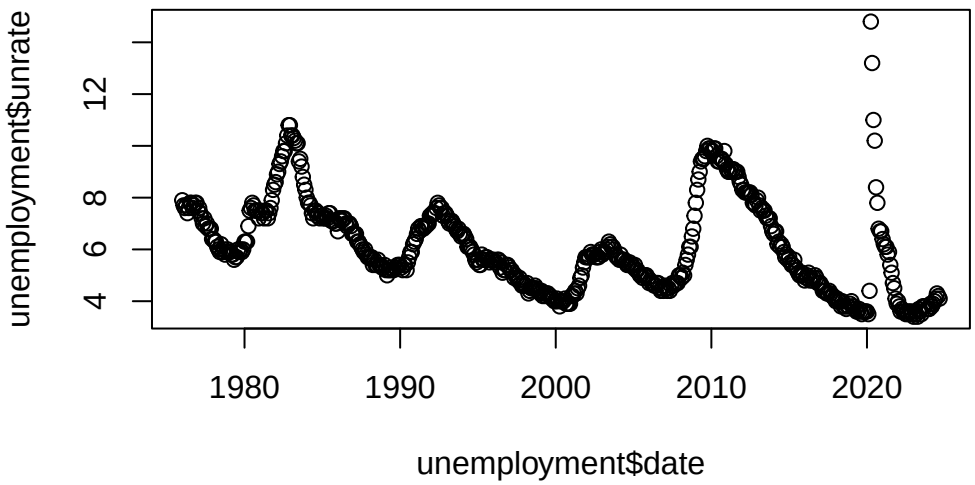
Let's introduce a new dataset on the rate of unemployment in the US. Sorry for

```r
unemployment <- read.csv("data/unemployment.csv")

# Convert `date` string into a `Date`:
unemployment$date <- ymd(unemployment$date)
```

Now, let's make a time-series plot of the unemployment rate over time

```r
plot(unemployment$date, unemployment$unrate)
```



Quarters

One important variable we might want is the quarter that a date falls within (Q1, Q2, Q3, and Q4). Let's try to make this using the `quarter` function from lubridate.

```r
# make new variable in unemployment called `quarter`
unemployment$quarter <- quarter(unemployment$date)
```

What does the argument `type = "year.quarter"` do? What is the difference?

```r
unemployment$quarter <- quarter(unemployment$date, type = "year.quarter")
```

year.quarter attaches the year to start. So quarter() outputs 1 for Q1, year.quarter outputs 1976.1 for Q1 1976.

Basic time-series regression

As a preview of what is to come, let's see which quarter of the year has the lowest unemployment rate:

```
library(fixest)
feols(
  unrate ~ 0 + i(quarter(date)),
  data = unemployment, vcov = NW() ~ date
)
```

```
OLS estimation, Dep. Var.: unrate
Observations: 585
Standard-errors: Newey-West (L=6)
                 Estimate Std. Error t value  Pr(>|t|)
quarter(date)::1  6.06259   0.221969 27.3127 < 2.2e-16 ***
quarter(date)::2  6.22993   0.259359 24.0205 < 2.2e-16 ***
quarter(date)::3  6.11361   0.226784 26.9579 < 2.2e-16 ***
quarter(date)::4  6.07222   0.227771 26.6593 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
RMSE: 1.75442   Adj. R2: -0.005435
```

Practice Questions

1. What day will it be in 37 days?

```
date(today + 37)
```

```
[1] "2026-05-21"
```

May 21st, 2026

2. How many days away is the fourth of July from today?

```
july4 <- date("2026-07-04")
today - july4
```

```
Time difference of -81 days
```

81 days away.

3. What day of the week is the fourth of July?

```
wday(july4, label = TRUE)
```

```
[1] Sat
Levels: Sun < Mon < Tue < Wed < Thu < Fri < Sat
```

Saturday

4. How many February 29th have their been since 1990?

```
dates <- paste0(1990:2026, "-02-29")

table(!is.na(ymd(dates)))
```

```
Warning: 28 failed to parse.
```

```
FALSE  TRUE
   28     9
```

There have been 9 Feb 29ths since 1990.

5. How many Friday the 13th have there been since 2000?

```
fri13 <- seq(date("2000-01-13"), date("2026-04-13"), by = "month")

classes_fri <- fri13[wday(fri13, label = TRUE) %in% c("Fri")]

length(classes_fri)
```

```
[1] 46
```

There have been 46 Friday the 13ths since 2000.